Let's also assume that the primary shards comprise a few million documents occupying 300 GB of memory. We therefore create a 10-shard index, with each shard allocated about 50 GB of memory. Elasticsearch distributes the 300 GB of data among 10 shards, so each shard gets about 30 GB of documents. We also have 2 replicas for each shard, for a total of 20 replicas.

Replicas need to be set up with the same memory as the main shard because they are copies of the shard. Thus, 20 replicas consume 20 times 50 GB, or 1000 GB. And don't forget to add the memory allocated to the primary shards: 10 shards times 50 GB is 500 GB. We need at least 1500 GB (1.5 TB) to run this cluster with *one* index. Figure 14.7 shows these calculations.

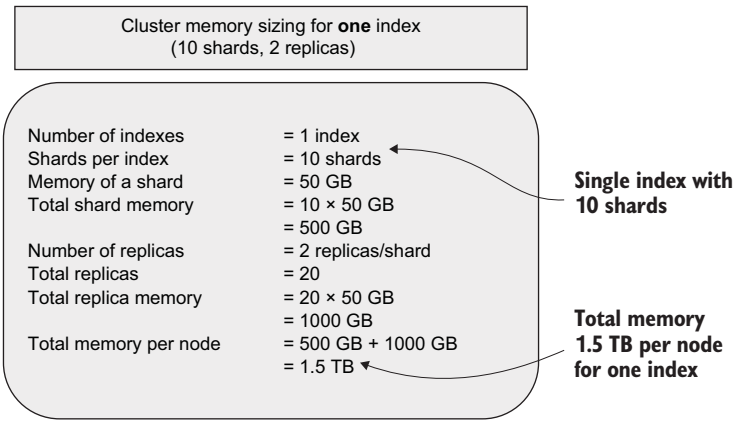


Figure 14.7 Memory sizing of a single index with 10 shards and 2 replicas

This cluster is made of individual nodes. Remember, these nodes need additional memory for operational purposes, including system indexes, in-memory data structures, and other operations. So, it is always advisable to add memory in addition to that allocated to shard sizing. As we mentioned, we are building a five-node cluster, so each node with 400 GB would make the cluster with 2000 GB. This will suffice for the current use case.

14.3.2 Setting up multiple indexes

In our example, we have only one index to manage, so we try to calculate memory costs based on that lone index. This is rarely the case in the real world. Any number of indexes can exist on the server, and at the very least, we need to provision servers to create several indexes in the future. If we extrapolate the cost calculations for five indexes, with each index having 10 shards and 2 replicas per index, figure 14.8 indicates our total memory.